BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

Bubble Sort

Name _____ **Class** _____ **Date** _____ **Time** _____ **Teacher** _____

Do Now - Four retrieval questions

BLACK PEN: answer four questions before checking the answers.

1. What must be true before binary search is used?

2. What does the MAR hold?

A) The last calculation result
B) The address being accessed
C) Data moving from memory
D) All startup instructions

3. "Functions can return only one digit and can never return an array." Correct this. Explain why.

4. A café program finds the cost of identical drinks. "Inputs are the final results." Correct this. Give the inputs, process and output.

Today's objectives

Explain the steps and required conditions for bubble sort. Apply the algorithm or method to supplied data.

Key vocabulary - Read, cover, recall

Bubble sort
Sorting by repeatedly comparing and swapping neighbouring values.

Pass
A sweep through adjacent comparisons.

Swap
Exchange the positions of two values.

Ascending order
Smallest to largest.

Write one key term and its meaning in your own words:

Do Now	I Do	We Do	You Do	Review & Improve
Recall previous learning	Read new knowledge	Practise with guidance	Work independently	Check my understanding

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

Learn and practise

New learning - Key ideas

Compare neighbours	Compare each adjacent pair in turn during a pass.
Swap when out of order	For ascending order, swap when the left element is greater than the right.
A complete pass	After one full left-to-right pass, the largest element in the unsorted part has bubbled right.
Repeat until sorted	Subsequent passes continue until no swaps are needed or the passes are complete.

I Do - One worked example

Show the array after each comparison in the FIRST pass of bubble sort on [5,2,8,1,4]. [5 marks]

1. Compare 5 and 2

2. Swap to [2,5,8,1,4].

3. Compare 5 and 8: no swap. Then compare 8 and 1: swap -> [2,5,1,8,4].

4. Compare 8 and 4: swap -> [2,5,1,4,8]. Largest value 8 is now at right. [5]

We Do - Guided questions

BLACK PEN: use the model, then explain your answer.

1. Show the array after ONE pass of bubble sort on [4,1,3,2]. [4]

2. Explain why [1,2,3,4] needs no swaps in an ascending bubble-sort pass. [3]

3. An array is [3,1,2]. Apply a complete pass and explain the result. [4]

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

Think and write

Code First - Predict > Write > Test > Improve

YOUR TASK: Correct the ascending-order comparison before a bubble-sort swap.

```
array sizes = [6,2]
if sizes[0] < sizes[1] then
temp = sizes[0]
sizes[0] = sizes[1]
sizes[1] = temp
endif
print(sizes[0])
```

BLACK PEN: predict or complete the code; test, then improve.

You Do - Independent exam questions

Q2. Give the array after one complete left-to-right bubble-sort pass on [7,2,5,1]. [4]

Q3. Explain why bubble sort compares adjacent items. [3]

Q4. State one way to detect that a bubble sort can finish early. [2]

Exit Ticket - True or False?

Circle T or F before reviewing Page 4.

1. Binary search can safely discard half only when data is sorted. ☐ T ☐ F
2. Compare each adjacent pair in turn during a pass. ☐ T ☐ F
3. For ascending order, swap when the left element is greater than the right. ☐ T ☐ F

BLACK PEN Answer all questions first. **GREEN PEN** Then: tick correct answers; rewrite mistakes.

Review & improve

GREEN PEN: tick correct marking points, then correct errors.

Choose your hardest question to rewrite in full below.

Do Now - Answers

Q1 The list must be sorted. This lets the search safely rule out half the remaining items. ☐

Q2 B - The address being accessed ☐

Q3 Yes. A subprogram can receive an array; a function can return an array. ☐

Q4 Inputs: price and quantity. Processing: multiply them. Output: total cost. Inputs come before the calculation; the output comes after it. ☐

Code First - Model

```
array sizes = [6,2]
if sizes[0] > sizes[1] then
temp = sizes[0]
sizes[0] = sizes[1]
sizes[1] = temp
endif
print(sizes[0])
```

Output: 2

We Do - Check the model

1. [1,4,3,2] [1]; [1,3,4,2] [1]; [1,3,2,4] [1]; 4 at right end [1]. ☐

2. Every left-hand element is already <= its neighbour [1]; no pair is out of order [1]; zero swaps in the pass [1]. ☐

3. 3>1 swap -> [1,3,2] [2]; 3>2 swap -> [1,2,3] [1]; largest element 3 at end [1]. ☐

Exit Ticket - Check

1. TRUE 2. TRUE 3. TRUE

You Do - Self-mark

Q2 7/2 swap [2,7,5,1] [1]; 7/5 swap [2,5,7,1] [1]; 7/1 swap [2,5,1,7] [1]; final [2,5,1,7] [1]. ☐

Q3 It checks whether each neighbouring pair is in order [1]; swaps an out-of-order pair [1]; repeated passes gradually put values into sorted positions [1]. ☐

Q4 Complete a pass with zero swaps [1]; array is already sorted [1]. ☐

Try the question again

1. Copy the FULL question you found most difficult:

2. In GREEN PEN, write a complete new answer: